

VeriRAG: A Knowledge Graph-Augmented RAG for Verilog and Assertion Generation

Jayanth Thangellamudi, Raghul Saravanan, and Sai Manoj Pudukotai Dinakarrao

Department of Electrical and Computer Engineering

George Mason University, Fairfax, VA, USA

Email: {jthangel, rsaravan, spudukot}@gmu.edu

Abstract—The adoption of Large Language Models (LLMs) in Electronic Design Automation (EDA) has demonstrated significant potential for automating Register-Transfer Level (RTL) generation and verification; however, conventional prompt-based or fine-tuned approaches often fail to produce structurally consistent RTL and meaningful assertions for complex designs. We present VeriRAG, a hybrid retrieval-augmented generation framework that combines hardware-specific knowledge graphs with semantic vector embeddings. This hybrid retrieval strategy provides both symbolic structural context and semantic content, enabling the LLM to generate synthesizable Verilog and valid SystemVerilog Assertions (SVAs) without relying on rigid manual intervention or costly retraining. Experimental results across a diverse set of representative designs show that VeriRAG achieves up to 97% syntax correctness and 100% functional success for RTL generation, with SVAs reaching 100% syntax validity and 95% Formal Property Verification (FPV) pass rates using standard EDA tools. These results highlight the potential of combining symbolic knowledge graphs with retrieval-augmented generation for scalable, verifiable hardware design workflows.

I. INTRODUCTION

Hardware Description Languages (HDLs), such as Verilog, VHDL, and SystemVerilog, are fundamental to modern integrated circuit (IC) and system-on-chip (SoC) design, providing precise RTL specifications. However, as design complexity scales, manual RTL designs remain labor-intensive and error-prone, driving the need for efficient automation in coding and verification, tasks that currently depend on simulation and formal methods, leading to substantial computational and scalability challenges [1], [2]. Traditional EDA tools partially address these challenges by automating tasks such as simulation and formal equivalence checking. Despite the efforts, RTL coding and assertion synthesis still demand significant manual effort and domain expertise, which limits scalability, coverage, and overall design productivity [3]. In particular, SVA generation remains complex and error-prone, often introducing semantic inconsistencies and human bias that degrade overall verification quality [4].

Recent advances in artificial intelligence (AI) and LLMs are transforming EDA workflows by automating tasks such as RTL synthesis, bug detection, and test generation [3]. Despite these advancements, the persistent hallucinations, where LLMs' outputs lack factual grounding or violate domain constraints, remain a critical barrier to their reliable hardware

design automation [5], and mitigating these is therefore essential to ensure that the generated RTL code aligns with synthesis and verification requirements. While domain-specific fine-tuning can partially address these challenges, due to its high computational and data costs, it limits rapid prototyping and iterative refinement.

Retrieval-Augmented Generation (RAG) has emerged as an effective method to enhance the factual accuracy and contextual relevance of LLM outputs by integrating external knowledge during generation. By dynamically retrieving domain-specific information from large repositories, RAG grounds responses in verifiable facts and mitigates knowledge gaps of standalone LLMs [6]. Over time, RAG systems have progressed from simple retrieval pipelines (Naïve RAG) to more advanced variants such as Modular RAG [7], which adapt dynamically to diverse domains. In complex and highly structured fields like hardware design, conventional text-based retrieval often falls short because it cannot capture the intricate relationships and dependencies inherent in RTL modules, signals, and parameters.

Building on this insight, we present a hybrid RAG framework, **VeriRAG**, designed to address persistent challenges in hardware design automation, particularly for RTL code and SVA generation, where structural integrity and semantic correctness are critical. Unlike conventional fine-tuned LLM pipelines [8]–[11], which often target narrow tasks and require repeated retraining, **VeriRAG** integrates knowledge graphs with semantic retrieval to ensure structural and functional consistency without costly fine-tuning. By systematically leveraging symbolic structure and semantic context, our framework improves generation quality, verification coverage, and adaptability, offering a robust alternative to static rule-based or isolated fine-tuning approaches.

The key contributions of this paper are:

- **To the best of our knowledge, we present VeriRAG, the first hybrid retrieval-augmented generation framework** that integrates hardware-specific knowledge graphs with semantic embeddings to automate RTL and SVA generation with explicit structural reasoning.
- VeriRAG introduces dynamic knowledge graph traversal and SPARQL querying to maintain cross-hierarchy structural integrity, fusing symbolic context into reusable prompt templates for inference. This ensures functionally correct RTL and SVAs without static rules or fine-tuning.

- On RTLLM benchmarks [12], VeriRAG achieves 97% syntax correctness and 100% functional accuracy for RTL, and 95% FPV pass rate for generated SVAs, outperforming baselines that handle these tasks separately.
- We demonstrate scalability by generating multi-module designs averaging over 1,000 RTL Lines of Code (LoC) and verifying them, and by producing 137 design-specific SVAs for an I2C circuit with a 95% FPV pass rate, validated with JasperGold.

II. BACKGROUND AND RELATED WORK

A. Overview of LLMs in EDA

The rapid advancement of LLMs has opened new directions for EDA. Originally developed for natural language tasks, their ability to process and generate code has sparked interest in automating hardware design. Recent works demonstrate that LLMs can translate plain-language specifications into syntactically correct Verilog code, generate SVAs, and automate testbench creation [5], reducing repetitive manual tasks.

Fine-tuning on curated RTL datasets aligns LLMs with domain-specific syntax and constraints; for instance, [8] showed that fine-tuning CodeGen on Verilog modules improved syntactic correctness by over 6% compared to the baseline. However, fine-tuning remains resource-intensive, requiring large annotated datasets, specialized hardware, and repeated retraining to adapt to evolving design patterns, limiting practicality for adaptive workflows.

To address these limitations, researchers are adopting retrieval-augmented approaches that combine LLMs' generalization capabilities with dynamic access to external domain knowledge. Unlike static fine-tuning, RAG frameworks enable LLMs to incorporate up-to-date design contexts without retraining, while grounding generation in factual, structured knowledge to mitigate hallucinations. This growing trend motivates the exploration of RAG strategies in EDA.

B. Retrieval-Augmented Generation (RAG) for EDA

RAG enhances LLMs by coupling model inference with dynamic access to external domain-specific knowledge. Instead of relying solely on static parameters, it retrieves relevant information from structured or unstructured repositories at inference time, preserving generalization while enabling flexible task adaptation without repeated fine-tuning. By grounding generation in retrieved factual data, RAG helps mitigate hallucinations and enhances output reliability [13].

Recent studies demonstrate RAG's effectiveness in software code tasks such as semantic retrieval and generation [13]. However, applying traditional embedding-only RAG to hardware workflows remains difficult due to the structured, hierarchical nature of RTL. Although emerging hardware-oriented RAG pipelines encode design hierarchies and verification constraints [13], they still rely primarily on vector similarity, which fails to capture the intricate signal dependencies, module boundaries, and control paths essential for synthesizability and functional correctness in RTL design.

To overcome these limitations, recent work [14] explores integrating symbolic representations with semantic embeddings to strengthen structural grounding during generation, motivating the use of knowledge graphs (KGs) for richer and more explicit modeling of hardware relationships.

TABLE I
COMPARISON OF VERIRAG WITH OTHER RTL/SVA FRAMEWORKS

Framework	Retrieval Method	Structural Support	Output Scope
VeriGen [8]	FT	Partial(modules, ports)	RTL Only
RTLCoder [15]	FT	Partial(modules, ports)	RTL Only
BetterV [9]	FT	✗	RTL Only
OriGen [16]	FT	✗	RTL Only
CodeV [17]	FT	✗	RTL Only
AssertLLM [11]	PE	signals	SVA Only
SANGAM [18]	PE	signals Only	SVA Only
VerilogCoder [19]	AG	signals Only	RTL Only
VeriRAG	RAG + KG	Full (modules, ports, signals, params, ops)	RTL + SVA

C. Knowledge Graphs (KG) for EDA

KGs are structured representations of entities and their relationships, modeled as labeled directed graphs. Initially developed for organizing information in domains such as web search and biomedical ontologies, KGs have been adopted in machine learning systems to provide external symbolic memory, enabling reasoning over structured relationships [14].

Unlike unstructured text or flat embeddings, a KG makes implicit design semantics explicit by formally encoding relationships such as `connected_to`, `drives`, or `depends_on`. This structural grounding enables both AI systems and human designers to query and reason about module hierarchies, signal interactions, and control logic that are otherwise difficult to infer from raw RTL alone. While KGs have been widely used for code comprehension in software, their use for structured hardware design remains underexplored, motivating our hybrid approach that unifies semantic similarity with symbolic KG traversal to generate synthesizable RTL and verifiable SVAs for modern hardware design flows.

D. Comparative Summary of Related Approaches

Table I summarizes how the proposed VeriRAG compares with recent fine-tuned (FT), prompt-engineered (PE), and agent-based (AG) frameworks for RTL and SVA tasks, focusing on retrieval, structural support, and functional scope.

Retrieval: FT models like VeriGen [8] and RTLCoder [15] embed domain knowledge in static weights, limiting adaptability to new modules. VeriRAG's hybrid strategy combines semantic similarity and symbolic reasoning, enabling the reuse of evolving design knowledge without costly re-training.

Structural Support: As summarized in Table I, many baselines have partial or no structural awareness, which VeriRAG addresses by encoding full hardware relationships to align RTL and SVAs with design constraints

Output Scope: Several frameworks automate either RTL or SVAs, but rarely both. Prompt-engineered tools like AssertLLM [11] or SANGAM [18] rely on rigid templates

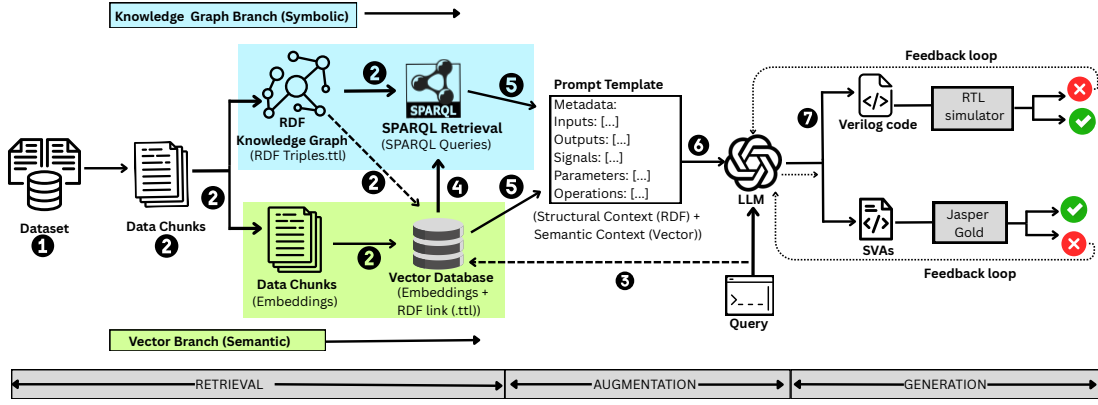


Fig. 1. Proposed VeriRAG workflow: A curated RTL dataset is split into chunks and processed via a knowledge graph branch (RDF triples) and a vector branch (embeddings). At inference, a user query triggers hybrid retrieval (vector + SPARQL), merging results into a prompt template for the LLM to generate synthesizable Verilog modules and SVAs, which are automatically verified for syntax and functional correctness, with failed outputs iteratively refined.

that miss complex logic. VeriRAG’s integrated pipeline co-generates RTL and SVAs, addressing gaps that cause incomplete verification in baselines.

These insights motivate our hybrid framework, VeriRAG, which systematically unifies semantic and symbolic retrieval to address the shortcomings of fine-tuning or vector-only generation. This integration provides a practical foundation for more reliable, context-aware hardware design automation.

III. PROPOSED FRAMEWORK: VERIRAG

To overcome the limitations of conventional LLM-based techniques in EDA, we introduce **VeriRAG**. This hybrid RAG-based framework integrates symbolic knowledge extracted from RTL sources with the generative capabilities of LLMs. VeriRAG is designed to automate two fundamental tasks in digital design: Verilog module generation and SVA generation.

An overview of the proposed VeriRAG framework is shown in Figure 1. The framework comprises three key stages: **retrieval**, **augmentation**, and **generation**, all supported by a curated Verilog dataset (detailed characteristics and statistics of the dataset are provided in Section IV) and a hybrid pipeline for context construction.

A. Data Processing and Knowledge Graph Construction

As shown in Figure 1 (1)–(4), the data processing pipeline begins by splitting the raw RTL dataset into manageable modules or chunks (1). For each module, VeriRAG uses two parallel branches, a symbolic knowledge graph branch and a semantic vector branch, that together create complementary representations for robust, context-aware retrieval.

In the knowledge graph branch (2), each RTL module is parsed to extract structural entities such as modules, ports, signals, parameters, and operations. These are then organized into a machine-interpretable graph using the Resource Description Framework (RDF), which stores each hardware fact as an explicit entity–relation–entity triple. While RDF has been widely applied for software call graphs and linked data, its use here is adapted specifically to hardware: it models hierarchical module instantiations, signal flows, parameterized widths, and operational dependencies that are critical for

preserving synthesizability and design integrity. This symbolic structure goes beyond generic graphs by encoding the unique entity–relation–entity patterns that underlie RTL semantics. Once encoded, each module’s RDF representation is stored as a .ttl file in the knowledge graph repository. SPARQL-based querying at inference time allows precise retrieval of these structural links, providing the LLM with consistent, design-specific context that pure embeddings cannot capture.

In parallel, the semantic vector branch (2) captures the semantic content of each module to complement the structural graph. Each chunk combines the raw RTL code, its parsed instruction, and an LLM-generated summary into a unified text representation. This summary bridges hardware-specific logic with natural language queries, improving prompt relevance during retrieval. The text is embedded as a high-dimensional vector that preserves semantic similarity across variations in design style. Crucially, each vector entry is stored as a **tuple** containing the embedding, instruction, RTL code, summary, and a direct link to its corresponding RDF graph file (stored as a .ttl file in the knowledge graph repository), which can be loaded dynamically for SPARQL queries, ensuring that semantic and symbolic contexts remain connected at inference.

Together, these two branches, one symbolic and one semantic, are stored in a persistent **hybrid database**. This design enables accurate, context-rich retrieval and helps ensure that the final RTL outputs remain both functionally meaningful and structurally consistent with hardware design principles.

B. Retrieval and Context Generation

The *query* is the natural language request provided by the user (e.g., “Generate a 32-bit ALU with add & subtract operations”), while the *prompt template* is the structured input assembled by the system, combining the query with retrieved semantic and structural context to guide the LLM generation.

VeriRAG employs a hybrid retrieval strategy that combines semantic similarity with structural precision. First, a vector-based retrieval process uses the query’s embedding (3) to efficiently identify top-k module matches using cosine similarity within the vector database. This enables the system to match user queries flexibly, regardless of variations in wording

or coding style, to relevant RTL chunks that contain similar functionality or design intent.

After the top-k modules are identified, the graph-based retrieval mechanism dynamically loads the RDF knowledge graph file associated with each matched module. SPARQL queries ④ are then executed to extract explicit structural details such as port lists, signal dependencies, module parameters, and operational hierarchies that purely semantic embeddings may not capture. This structural context ensures that any retrieved RTL design remains consistent with its hardware-specific constraints and design topology.

LLM Prompt Template (Excerpt)

```
# Verilog Module Generation
**Query**: Generate an ALU with add/subtract
/AND/OR/XOR operations.
**Context**:
Module: alu_basic
Instruction: Simple ALU with 8-bit inputs.
Summary: Supports arithmetic and logic
operations.
Code: ...
Metadata:
  Inputs: [...]
  Outputs: [...]
  Signals: [...]
  Parameters: [...]
  Operations: [...]
**Task**:
- Generate a standalone, synthesizable
  Verilog module.
- Use localparam for opcodes.
- Inline SystemVerilog Assertions if
  requested.
- Output in a single code block.
```

Fig. 2. Example LLM prompt template for Verilog module and SVA generation.

The results of both retrieval paths are integrated through a prompt assembly step ⑤, which merges the natural language summaries, code fragments, and RDF-derived triples into a single, reusable prompt template as shown in Figure 2. This hybrid retrieval mechanism ensures that even when a query specifies unfamiliar module names or operations, the semantic branch flexibly matches conceptually similar designs, while the symbolic graph branch enforces structural consistency and valid design constraints. Together, this hybrid context equips the LLM with rich, multi-granular information, enabling it to generate synthesizable Verilog modules and aligned SVAs that satisfy both functional requirements and hardware correctness, even under ambiguous query formulations.

C. Code Generation and Verification

Once the final prompt is assembled ⑤ by merging semantic matches from the vector store and precise structural context from the RDF knowledge graph via SPARQL queries ⑤, the LLM ⑥ generates RTL modules and SVAs ⑦ that align with the user's functional intent and hardware constraints. This hybrid of symbolic structure and semantic embeddings ensures the generated design preserves module hierarchies, signal dependencies, and standard hardware conventions.

To ensure that each generated output is syntactically valid and functionally correct, all RTL modules and SVA are automatically validated using industry-standard EDA tools. We run RTL simulation using Cadence Xcelium to verify syntax and functional behavior, while property checks on SVAs are formally verified with JasperGold.

In addition, the framework incorporates a lightweight feedback loop: if any generated module or SVA fails simulation or formal checks syntactically, the failed result is flagged and fed back to the LLM as shown in Figure 1. This iterative refinement enables the system to correct inconsistencies dynamically without requiring costly retraining. Importantly, this feedback mechanism operates purely at the retrieval and prompt assembly level, with no updates to LLM parameters, thus avoiding the high computational cost and domain rigidity of traditional fine-tuning. In practice, the feedback loop converges within 1–3 iterations for most designs, and we enforce a maximum of 5 iterations per query to prevent indefinite retries, although this upper limit was never reached in our experiments. By combining hybrid retrieval, structured prompt construction, and iterative validation, this flow ensures that the final outputs are accurate and compliant with standard design practices.

This comprehensive flow from hybrid retrieval to prompt assembly and rigorous verification demonstrates the framework's practicality and reliability for real-world hardware tasks. The next section details experimental results that illustrate the framework's effectiveness on diverse RTL benchmarks.

IV. EXPERIMENTAL RESULTS

A. Dataset Details

The dataset comprises approximately 160 synthesizable Verilog modules, carefully curated to provide a robust foundation for benchmarking RTL generation and verification pipelines. The corpus is composed of the following sources:

a) Open-source Corpus and Educational Snippets: The dataset includes modules from open-source repositories like OpenCores [20] and RTL GitHub projects, filtered for valid '.v' files with complete *module–endmodule* pairs and reasonable size. To broaden coverage, we added modules from standard HDL textbooks [21], [22] and verified educational IP blocks [23], cleaned and extracted for clarity. This combination provides a robust foundation for realistic RTL benchmarking.

b) Design Diversity and Scale: The final corpus spans:

- Basic combinational designs (e.g., muxes, encoders)
- Sequential blocks (e.g., counters, shift registers)
- Small subsystems (e.g., RAM, FIFO, UART)
- Larger SoC cores (e.g., RISC-V modules, matrix ops)

Designs range from 10-300 lines, with approximately 40% parameterized. Overall, approximately 45% of modules are purely combinational, 40% sequential, and 15% mixed or protocol-driven. The dataset covers diverse modules to benchmark RTL and SVA generation pipelines.

B. Evaluation Setup and Baselines

We evaluate VeriRAG using the RTLLM benchmark [12], which includes 30 RTL designs covering simple combinational

TABLE II
COMPARISON ACROSS 30 RTLLM DESIGNS, INCLUDING TESTBENCH-VERIFIED FUNCTIONAL ACCURACY AND ASSERTION CHECKS

Design	GPT-3.5 [24]		GPT-4 [24]		Thakur et al. [8]		StarCoder [10]		VeriRAG RTL			VeriRAG SVA	
	Syntax	Func	Syntax	Func	Syntax	Func	Syntax	Func	Syntax	Func	LOCs	#Gen/Syntax/FPV	
accum	4	✓	5	✓	0	-	0	-	5	✓	37	6/6/6	
adder	4	✓	5	✓	0	-	0	-	5	✓	27	8/8/7	
Adder_16bit	5	✗	5	✓	5	✓	0	-	5	✓	41	10/10/8	
Adder_32bit	5	✗	5	✗	0	-	0	-	5	✓	43	8/8/8	
Adder_64bit	2	✗	3	✗	0	-	0	-	4	✓	57	8/8/8	
multi_8bit	3	✗	4	✗	0	-	0	-	4	✓	24	6/6/5	
multi_16bit	0	-	5	✓	5	✓	0	-	5	✓	61	7/7/7	
multi_pipe_4bit	0	-	2	✓	0	-	5	✓	4	✓	30	8/8/8	
multi_pipe_8bit	0	-	4	✗	0	-	5	✓	5	✓	33	5/5/5	
div_8bit	0	-	0	-	0	-	0	-	5	✓	44	2/2/1	
div_16bit	0	-	5	✗	0	-	0	-	5	✓	56	20/20/18	
JC_counter	5	✓	5	✓	5	✗	5	✗	4	✓	22	4/4/4	
right_shifter	5	✓	5	✓	5	✓	0	-	5	✓	15	10/10/9	
mux	0	-	4	✓	5	✓	0	-	5	✓	24	7/7/7	
counter_12	5	✓	5	✓	5	✗	5	✗	5	✓	19	7/7/7	
freq_div	5	✗	5	✗	5	✗	0	-	5	✓	44	5/5/5	
signal_generator	5	✓	5	✓	0	-	5	✗	5	✓	37	4/4/4	
serial2parallel	5	✓	5	✓	0	-	5	✗	5	✓	38	16/16/16	
parallel2serial	5	✗	4	✗	0	-	0	-	5	✓	33	9/9/9	
pulse_detect	1	✗	5	✗	5	✗	0	-	5	✓	59	16/16/16	
edge_detect	5	✓	5	✓	5	✗	0	-	5	✓	26	5/5/5	
FSM	5	✗	5	✗	5	✗	0	-	5	✓	48	9/9/8	
width_8to16	5	✓	5	✓	0	-	5	✓	5	✓	41	7/7/7	
traffic_light	5	✗	5	✓	0	-	0	-	5	✓	104	4/4/4	
calendar	0	-	5	✗	0	-	0	-	5	✓	47	6/6/6	
RAM	0	-	0	-	5	✓	5	✓	5	✓	34	5/5/5	
asyn_fifo	0	-	0	-	0	-	0	-	5	✓	74	8/8/8	
ALU	0	-	5	✗	0	-	0	-	5	✓	46	13/13/12	
PE	4	✓	5	✓	5	✗	5	✓	5	✓	20	8/8/6	
risc_cpu	0	-	0	-	0	-	0	-	5	✓	38	9/9/9	
Success rate	55%	10/30	81%	15/30	40%	5/30	27%	5/30	97%	30/30	-	240/240/229	(100%/95%)

circuits, datapaths, and SoC-level subsystems. This diversity ensures the framework’s performance generalizes across varying design complexities. Following prior work [8], [12], we adopt the *pass@k* metric ($k = 5$), defined as:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \quad (1)$$

where n is the number of generated samples and c is the number that passes both syntax and simulation checks.

Our baselines include state-of-the-art RTL generation works, such as GPT-3.5 [24], GPT-4 [24], the fine-tuned VeriGen [8], and StarCoder [10]. To ensure fair comparisons, we use Cadence Xcelium for RTL simulation and JasperGold for SVA functionality checking.

While our baselines rely on task-specific fine-tuning, this comparison illustrates that VeriRAG achieves comparable or superior domain accuracy by integrating symbolic knowledge graphs with semantic embeddings at inference time. This result underscores that retrieval-augmented strategies can deliver high-quality, contextually grounded outputs while avoiding the rigidity and challenges of repeated fine-tuning.

The framework’s reproducible implementation leverages Pyverilog for RTL parsing, RDFlib for RDF graph construction, and ChromaDB for semantic embeddings generated with OpenAI’s *text-embedding-3-small* model. Prompt generation, summarization, and final code output use GPT-4, with Tiktoken ensuring compliance with input limits. This modular pipeline ensures that VeriRAG can be adapted to other hardware design flows with minimal reconfiguration.¹

C. Generation and Verification

Table II summarizes the quantitative evaluation of both syntax and functionality correctness of all four evaluated

¹The full VeriRAG implementation, dataset, and scripts are available on <https://github.com/HArt-Git/VeriRAG>

LLMs using RTLLM benchmarks [12]. The syntax part counts the number of generated design RTLs with correct syntax, out of the five trials. Then, the functionality part (i.e., Func.) will count a success (✓) as long as at least one RTL file is successfully generated and passes the testbench, among the codes already with correct syntax. For SVAs, both syntax correctness and functionality are verified formally using JasperGold; functionality (i.e., Func.) is measured by the FPV pass rate.

1) *RTL Generation and Verification*: Unlike standard LLM pipelines, VeriRAG enriches each prompt with symbolic structural information from a hardware-specific KG. This allows the model to capture module hierarchies, port connections, and signal dependencies that are critical for generating contextually accurate, synthesizable RTL. As summarized in Table II, VeriRAG achieves a syntax correctness of 97% and is also 100% functionally correct in all the test cases, outperforming GPT-3.5 (syntax: 55%, func: 33%) and GPT-4 (syntax: 81%, func: 50%). Notably, the framework succeeds on designs that baselines often fail to synthesize or simulate correctly, such as *multi_pipe_8bit*, *div_8bit*, and the *risc_cpu*, all of which reach correct syntax and pass simulation with VeriRAG. Unlike prior works, VeriRAG produces functionally and syntactically correct RTL in nearly all cases across five trials, demonstrating robust reliability. These results highlight the importance of the knowledge graph in producing RTL that is not only syntactically valid but also functionally accurate.

2) *SVA Generation and Verification*: Unlike approaches that rely on separate verification datasets, VeriRAG produces SVAs purely from a structured prompt template enriched with the symbolic metadata from the knowledge graph, with no dedicated assertion database, which highlights the strength of structural info through KGs. The KG encodes port types, signal flows, and state relationships, giving the LLM enough

structured context to generate meaningful, design-specific checks automatically.

We evaluate SVA generation on the same RTLLM benchmarks to ensure that each assertion directly targets the generated RTL modules under test, maintaining functional alignment. As shown in Table II, the framework achieves 100% syntax correctness and 95% functionally correct in all the test cases, significantly outperforming baselines like [8] and [10], which lack built-in assertion generation. This integrated approach demonstrates how symbolic retrieval makes the framework more robust: even for modules like *Adder_64bit* and *parallel2serial*, which contain subtle signal interactions, VeriRAG generates assertions that pass formal checks on the first attempt—an outcome rarely achieved by baselines without manual fixes.

TABLE III

DESIGNS WITH CORRESPONDING MODULES AND LINES OF CODE (LoCs)

Design	Modules	LoCs
Network-on-Chip (NoC)	Buffer_Unit, Network_Interface, Node, Router, Routing_Unit, Switch_Allocator, Switch, Traffic_Generator, top	1018
AI Accelerator	Adder_Block, Systolic_Array, AXI_Interface, Data_Feeder, dma_controller, MA_Block, Multiplier_Block, Processing_Element, PSM_Core, RAM_Block, top	1389
Multimedia Processing Unit	AXI_Interconnect, Cache_Controller, Video_Decoder, Display_Controller, DMA_Controller, DSP_Core, Image_Processor, Memory_Controller, Frame_Buffer_Controller, top	1310
PCIe Root Complex	TLP_Generator, TLP_Buffer, Root_Port_Arbitrer, PCIe_Link_Training, MSI_X_Controller, Config_Space_Manager, AXI_Bridge, Address_Translation_Unit, top	1012

D. Scalability and RTL/SVA Generation Evaluation

To demonstrate the scalability of **VeriRAG** for both RTL and SVA generation, we conducted two targeted evaluations.

1) *RTL Generation Quality*: To rigorously evaluate the scalability of VeriRAG’s RTL generation capabilities, we selected four diverse multi-module design examples ranging from high-throughput interconnects to complex accelerators and controllers (Table III). Each design comprises between 8 and 11 interconnected modules with total lines of code (LoCs) spanning from approximately 1,000 to nearly 1,400 lines.

These designs include typical building blocks such as network buffers, DMA controllers, PCIe bridges, AXI interconnects, cache controllers, and processing elements — all of which require precise module instantiation, parameterization, and consistent signal wiring across hierarchy levels. VeriRAG successfully generated all RTL modules with 100% syntax correctness and functional simulation pass rates, verifying that even designs with deep hierarchies and multiple communication interfaces can be produced reliably from the hybrid symbolic–semantic context.

These result confirms that VeriRAG’s hybrid retrieval scales reliably from small single-purpose modules to realistic multi-module designs with thousands of lines of code, demonstrating its practical viability for real-world digital design automation.

2) *SVA Generation Quality*: We further analyzed VeriRAG’s assertion generation scalability by benchmarking it against AssertLLM [11] and GPT-4 [24] on a diverse set of signals from an I2C controller design, grouped by signal type.

TABLE IV
EVALUATION OF THE GENERATED SVAS FOR DESIGN I2C

Signal Type		Signal Name	AssertLLM [11]	GPT-4 [24]	VeriRAG
Assertion Evaluation (# Generated/# Syntax Correct/# FPV Pass)					
I/O Port	Clock	wb_clk_i	1/1/1	3/1/0	4/4/4
	Reset	wb_rst_i	1/1/1	3/1/0	13/13/12
		arst_i	1/1/1	3/1/0	10/10/9
	Control	wb_stb_i	3/3/2	3/1/0	9/9/9
		wb_ack_o	2/2/1	3/1/0	5/5/5
		wb_inta_o	2/2/1	3/1/0	5/5/5
		wb_adr_i	1/1/1	3/1/0	6/6/6
	Data	wb_dat_i	1/1/1	3/1/0	7/7/7
		wb_cyc_i	1/1/1	3/1/0	6/6/6
		wb_dat_o	1/1/1	3/1/0	6/6/6
		wb_we_i	1/1/1	3/1/0	6/6/5
		scl_pad_i	1/1/1	3/1/0	5/5/3
		scl_pad_o	1/1/1	3/1/0	8/8/8
		sda_pad_i	1/1/1	3/1/0	6/6/6
		sda_pad_o	1/1/1	3/1/0	6/6/6
		scl_pad_oe	1/1/1	3/1/0	5/5/4
		sda_pad_oe	1/1/1	3/1/0	6/6/6
Register	control	ctr	10/10/10	3/1/1	4/4/4
		sr	15/15/14	3/1/1	6/6/6
	Data	prer	4/2/2	4/1/1	3/3/2
		tx	2/2/2	3/1/1	4/4/4
		rx	2/2/2	3/1/1	4/4/4
		cr	2/2/2	4/1/1	3/3/3
Design Total		56/56/50 (89%)	71/23/6 (8%)	137/137/130 (95%)	

Table IV shows that VeriRAG generates significantly more assertions per signal while maintaining high correctness.

Compared to AssertLLM [11] and GPT-4 [24], which produce only 1–3 assertions per signal with high failure rates during formal property verification (FPV), VeriRAG generates up to 13 assertions per reset signal and over 15 for complex registers. Importantly, of the 137 generated assertions, all passed syntax checks and 95% passed FPV, demonstrating robust design alignment and coverage. This highlights VeriRAG’s ability to scale assertion generation both in volume and quality, enabling more comprehensive hardware verification compared to prior works.

Overall, these results demonstrate that VeriRAG scales robustly to complex multi-module RTL generation while providing high-coverage, formally verified SVAs, outperforming prior works that treat RTL and assertion generation separately.

V. CONCLUSION

This paper presents VeriRAG, a structured retrieval-augmented generation framework that integrates a hardware-specific knowledge graph with LLM-based RTL and SVA generation. By combining symbolic structural retrieval with semantic embeddings, the framework achieves robust syntax and functional correctness for both Verilog modules and SystemVerilog Assertions, consistently outperforming conventional fine-tuned and prompt-engineered baselines. Quantitative results show that VeriRAG achieves 97% syntax correctness and 100% functional simulation pass rates for RTL generation across RTLLM benchmarks, while the SVAs reach 100% syntax validity and 95% formal property pass rates when verified with JasperGold. Furthermore, the framework scales effectively to multi-module designs, averaging over 1,000 LoCs, maintaining full syntactic and functional correctness. On the I2C design case study, VeriRAG generated 137 SVAs, with 95% passing formal checks, significantly exceeding the coverage of GPT-4 and AssertLLM baselines. These results demonstrate that integrating symbolic structural context significantly improves the reliability and verifiability of LLM-based hardware design automation.

REFERENCES

- [1] S. Ray, E. Peeters, M. M. Tehranipoor, and S. Bhunia, "System-on-chip platform security assurance: Architecture and validation," *Proceedings of the IEEE*, vol. 106, no. 1, pp. 21–37, 2018.
- [2] R. Saravanan and S. M. Pudukotai Dinakarrao, "The fuzz odyssey: A survey on hardware fuzzing frameworks for hardware design verification," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 192–197.
- [3] K. I. Gubbi, S. A. Beheshti-Shirazi, T. Sheaves, S. Salehi, S. M. PD, S. Rafatirad, A. Sasan, and H. Homaoun, "Survey of machine learning for electronic design automation," in *Proceedings of the Great Lakes Symposium on VLSI 2022*, ser. GLSVLSI '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 513–518.
- [4] R. Kande, H. Pearce, B. Tan, B. Dolan-Gavitt, S. Thakur, R. Karri, and J. Rajendran, "(security) assertions by large language models," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4374–4389, 2024.
- [5] K. Xu, R. Qiu, Z. Zhao, G. L. Zhang, U. Schlichtmann, and B. Li, "Llm-aided efficient hardware design automation," 2024.
- [6] G. Xiong, E. Xie, C. Williams, M. Kim, A. H. Shariatmadari, S. Guo, S. Bekiranov, and A. Zhang, "Toward reliable scientific hypothesis generation: Evaluating truthfulness and hallucination in large language models," 2025.
- [7] Y. Gao, Y. Xiong, M. Wang, and H. Wang, "Modular rag: Transforming rag systems into lego-like reconfigurable frameworks," 2024.
- [8] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, Apr. 2024.
- [9] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betternv: controlled verilog generation with discriminative guidance," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [10] R. Li, L. B. allal, Y. Zi, N. Muennighoff, D. Kocetkov, and C. M. et al., "Starcode: may the source be with you!" *Transactions on Machine Learning Research*, 2023.
- [11] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, Z. Xie, and H. Zhang, "AssertLLM: Generating and evaluating hardware verification assertions from design specifications via multi-llms," 2024.
- [12] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLML: An open-source benchmark for design RTL generation with large language model," in *Asia and South Pacific Design Automation Conference*, 2024.
- [13] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," 2024.
- [14] B. Subagdja, D. Shanthoshigaa, Z. Wang, and A.-H. Tan, "Machine learning for refining knowledge graphs: A survey," *ACM Comput. Surv.*, vol. 56, no. 6, Feb. 2024.
- [15] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," in *2024 IEEE LLM Aided Design Workshop (LAD)*, 2024, pp. 1–5.
- [16] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, Y. Liang, X. Zhang, D. Song, and D. Lin, *OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection*. New York, NY, USA: Association for Computing Machinery, 2025.
- [17] Y. Zhao, D. Huang, C. Li, P. Jin, M. Song, Y. Xu, Z. Nan, M. Gao, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang, X. Zhang, Z. Du, Q. Guo, and X. Hu, "Codev: Empowering llms with hdl generation through multi-level summarization," 2025.
- [18] A. Gupta, B. Mali, and C. Karfa, "Sangam: Systemverilog assertion generation via monte carlo tree self-refine," 2025.
- [19] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool," 2025.
- [20] OpenCores, "OpenCores: Open Source Hardware Community," <https://opencores.org/>, 2025, accessed: 2025-07-08.
- [21] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 4th ed. McGraw-Hill Education, 2020.
- [22] S. Palnitkar, *Verilog HDL*, 2nd ed. Prentice Hall, 2003.
- [23] Massachusetts Institute of Technology, "6.111 Introductory Digital Systems Laboratory," <https://ocw.mit.edu/courses/6-111-introductory-digital-systems-laboratory-spring-2006/>, 2006, accessed: 2025-07-08.
- [24] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.